

2026 MITRE eCTF Design Document



February 25, 2026

Overview.....	3
Team Overview.....	3
Terminology and Notation.....	3
System Description.....	3
System Architecture.....	3
Security Requirements.....	4
System Design.....	4
Global Secrets.....	4
Functional Requirements.....	4
Security Design.....	7
Cryptographic Protocols.....	7
Encryption and Decryption.....	7
Defenses.....	8
Buffer Overflow Protections.....	8

Overview

This design document describes Tufts University's implementation of a Hardware Security Module system. Our design employs a multi-layer security approach combining hardware-optimized cryptography via WolfSSL and robust key management. Our implemented security principles include:

Encryption: All files created or stored on HSMs and all HSM to HSM communications containing sensitive information are AES encrypted.

Authentication: No sensitive information on the HSM can be accessed without passing pin and permission verification and possessing the predetermined group-specific key for the permission category.

Forward Secrecy: Ephemeral keys for each group and permission type are utilized to ensure forward secrecy

Team Overview

Advisors:

Steven Bell & Ming Chow

Students:

Patrick Johnson, Kalan Brunell, Joe-Ansel Puplava, Jet Yotsuuye & Nathan Edwards

Terminology and Notation

HSM: Hardware Security Module

ECC: Elliptic Curve Cryptography

AES: Advanced Encryption Standard

FAT: File Allocations Table

System Description

System Architecture

The HSM system consists of the following main components

- **Host Computer:** Issues functional commands to HSMs
- **HSMs:** Stores and transfers files
- **Management Interface:** Allows communication between Host Computer and HSMs
- **Transfer Interface:** Allows communication between HSMs

The Host Computer sends a command to an HSM using the Management Interface, and if that command requires files be sent to or received from another HSM, the two HSMs communicate over Transfer Interface. When a command is concluded, a success message, along with any accompanying data, will be sent back to the Host Computer.

Security Requirements

Requirement 1:

An attacker should not be able to perform any file action without a validly provisioned HSM with the permissions to perform that action on files belonging to that group.

Requirement 2:

No PIN-protected action should be able to be completed by a user without prior knowledge of the PIN

Requirement 3:

An HSM should not successfully receive any file that was not generated by another valid HSM with write permissions for that group

System Design

Global Secrets

Our global secrets file will contain asymmetric keys for every permission group. For each permission group, there are 2 types of asymmetric keys.

1. The first type of asymmetric keys is for reading and writing files on the HSMs. An HSM will receive the decryption key if it has permission to read files from that group, and an HSM will receive the encryption key if it has permission to write files for that group. We will address these keys as the read key and write key, respectively, throughout this document.
2. The other type of asymmetric keys are for checking if an HSM has receive permissions for a specific file group. The private key will be used to sign a verification message back to the listening device to make sure that the HSM has receive permission for that file group. The public key will then be used to check if the signature is accurate. We will address these keys as the verify key and check key, respectively, throughout this document.

Additionally, global secrets will contain two symmetric keys for the receive and interrogate command. They are used to verify that they got the command from a valid HSM.

Functional Requirements

List Command:

Given a valid pin, the list command returns a packet containing the metadata for all files on the HSM to the host.

Write Command:

Given a valid pin, here are the steps that the write command takes to encrypt and store a file:

- The HSM will check if it has the write permissions for the given file group.
- Then we generate an AES symmetric key and initialization vector (IV) using wolfcrypt's random function. We then encrypt the file using the key and IV.
- We then derive the write key of that specific file group from the secrets.h file (which is the ECC public key) and encrypt the AES key. This ensures that the only HSM's that can decrypt the AES key is an HSM with the corresponding read permission for that file group.
- Finally, we append the encrypted AES key onto the front of the file and store it into flash memory.

Read Command:

Given a valid pin, here are the steps that the write command takes to decrypt and read the file:

- The HSM first checks to see if the HSM has read permissions for the file group that is going to be read. If they don't have permissions, error out.
- The HSM takes the header and decrypts using both the ECC write and ECC read key of that file group (Wolfcrypt requires both public and private key to decrypt) in order to retrieve the AES key
- Once the AES key is retrieved, it decrypts the message using that AES key.
- Finally, once decrypted, it writes out to the designated output file.

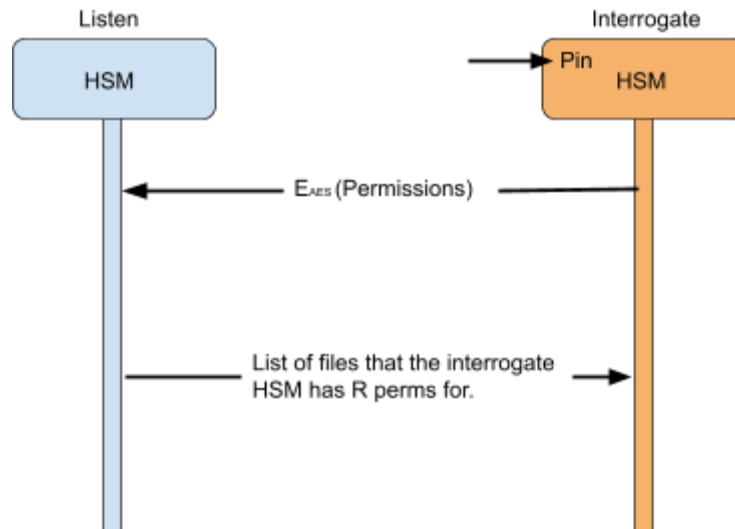
Listen Command:

The listen command allows the HSM to receive the interrogate or receive command from another HSM. The functionality of responding to the interrogate and receiving commands is handled by the listen command via case statements.

Interrogate Command:

Given a valid pin, the interrogate command allows an HSM to receive a packet containing all the metadata for all files on another HSM:

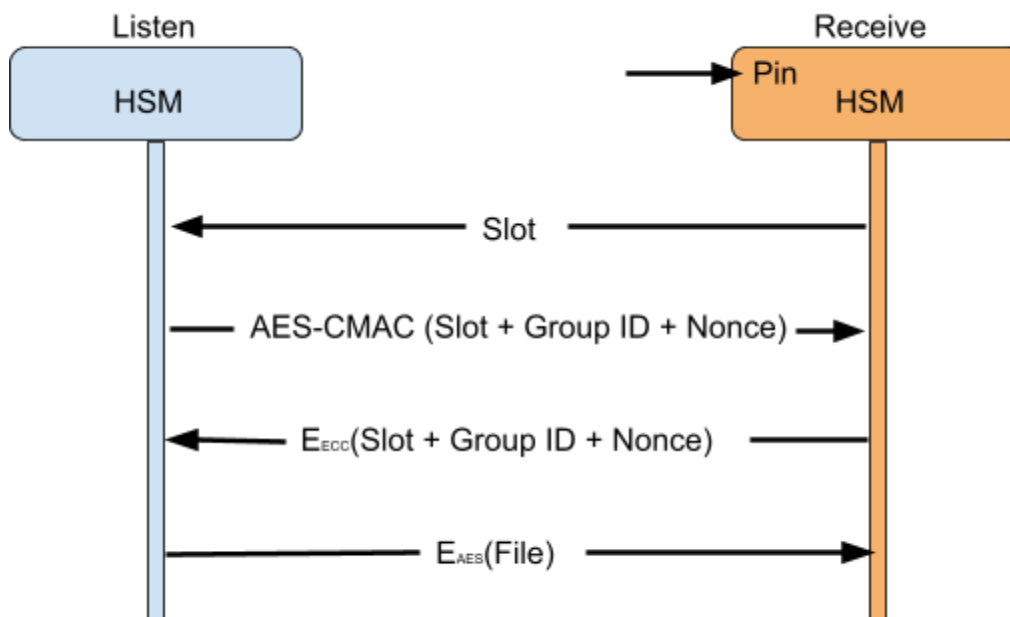
- The interrogating HSM sends over a list of all of the Group Permissions, encrypted with AES.
- The Listening HSM will then list the files only for which it has valid Receive permissions and sends it back to the interrogating HSM



Receive Command:

Given the correct PIN, the receive command runs a bit differently from the other commands. To protect against HSMs receiving files that are in file groups that they do not have receive permissions for, we implemented a 2-step handshake protocol that protects against replay attacks and permission changes during communication:

- First, the receiving HSM sends a packet containing the slot that it wants to receive from. The receive HSM remains in a listening state for the challenge
- The listening HSM, when receiving the packet, creates a struct with information about the slot, the file group ID of the file that is in the slot, and a random nonce that is created at runtime. We then sign that struct with AES-CMAC and send the challenge to the receiving HSM. The listening HSM remains listening.
- When the receiving HSM gets this packet, it checks it, and breaks down the struct. The receiving HSM then sees if they have the receive permissions for the given file group:
 - If it does not have permission, it will reject the request and send an abort message to the listening device, saying to abort the interaction.
 - If it does have permission, it will find the receive key (private ECC key) for that file group in secrets.h and sign the slot, group id, and nonce with the key. It will then send that signature to the listening HSM
- The listening HSM will check the signature with the check key (public ECC key) and compare the information with the information it sent before. If everything lines up, then it sends the file encrypted with AES (along with any necessary information) over to the receiving HSM.



Security Design

Pin Checking

Any command except for Listen requires a pin. The pin for a particular HSM is stored as a hash using SHA-256 in Secrets.h. We check that the pin is correct by doing a constant-time comparison algorithm on a hash of the pin; that way, if flash is leaked, the attacker will only have access to the hash and not the pin itself.

Additionally, to prevent brute-force, we add a 3-second cooldown for every pin attempted to be input. We attempted to flash the timer to memory; however, we could not get it to work. Doing some testing, we found that it would be virtually impossible to brute-force attack the pin using power-cycling to get around the cooldown because of the natural delay of the board's startup.

Finally, to prevent teams from using side-channel analysis on our PIN (for example, if we use a for loop to compare letters and have it stop after it finds the letter, they can calculate how long it takes to compare the pin and how long it takes to for each loop and deduct what the letters of the pin are), we use a constant time compare function that checks every byte of the pin no matter what.

Cryptographic Protocols

We use two different cryptographic protocols in our design, ECC and AES.

- ECC is an asymmetric key encryption algorithm, meaning it has a Public/Private key pair

- AES is a symmetric key encryption algorithm and uses only a private key and the private key must be stored securely.

Encryption and Decryption

We use AES in CBC mode for encrypting files in the write command, decrypting files in the read command, and transferring permissions in interrogate. For read and write operations, AES allows us to encrypt large files within the given time requirements. For interrogate, AES allows us to use a single key for all HSMs in the system.

We use AES-CMAC for the signing process. This is because CMAC is faster than CBC, which we need given the 2-step process for receive.

We use ECC primarily as a way to ensure that HSM's without certain permissions are physically unable to perform those actions because they do not have the keys to perform those actions. This applies to reading/writing and signing/checking. HSM's with only the public key cannot derive the private key, which enforces security for HSM's that only have write and check permissions for a group. However, we do note that HSM's with read and verify permissions will have the private and public key. This is due to the fact that ECC public keys are derived from the private key. So if a private key is obtained, then the corresponding public key could be derived.

Defenses

Buffer Overflow Protections

Both functions for writing and reading packets over UART have a parameter for the maximum length. For reading packets, the maximum length is explicitly defined as the size of the buffer, so that it will never receive more data than it expects, protecting against buffer overflows. For writing packets, the maximum length is also based explicitly on the size of the buffer so that we can't overflow our own buffer on accident. Additionally, slot numbers are checked for all command calls that use them to avoid FAT overflow.

UART was also changed to be more explicit in which channel it selects so that data is not printed to a computer when it is meant to be sent to another HSM or vice versa. Defense against Side-Channel Attacks was added for pin checking with the implementation of a lockout counter that persists even through power cycling.